Android To-Do List Application Abstract

Anas Alaa

Ain Shams University, Cairo, Egypt, 2023170106@cis.asu.edu.eg

Razan Saied

Ain Shams University, Cairo, Egypt, 2023170218@cis.asu.edu.eg

Ahmed Mohamed

Ain Shams University, Cairo, Egypt, 202317058@cis.asu.edu.eg

Omar Mohamed

Ain Shams University, Cairo, Egypt, 2023170396@cis.asu.edu.eg

Ahmed Mohamed

Ain Shams University, Cairo, Egypt, 20201700079@cis.asu.edu.eg

Managing everyday tasks on a mobile device calls for an interface that is fast, reliable, and usable without a network connection. This paper presents a native Android to-do list application that lets users create, view, edit, delete, filter, and share tasks. The application is written in Kotlin and follows the Model–View–ViewModel (MVVM) architecture. Its declarative interface is built with Jetpack Compose and Material Design 3 and adapts to different screen sizes. Task data is stored locally with the Room persistence library, which provides complete CRUD operations and reactive updates through Kotlin Flow. Navigation Compose connects the screens, and an implicit Intent lets users share tasks. The project was developed by a team of five students and managed through GitHub.

CCS CONCEPTS • Human-centered computing → Ubiquitous and mobile computing • Software and its engineering → Software design engineering

Additional Keywords and Phrases: Android, Kotlin, Jetpack Compose, Room, MVVM, mobile application development

1 INTRODUCTION

To-do list applications are among the most widely used productivity tools on mobile devices. A good implementation must respond instantly, keep data safe between sessions, and adapt to different screen sizes. This project builds such an application as an exercise in modern Android development.

Its goals are to apply core Android concepts (Compose, Intents, navigation), persist data with Room, write clean Kotlin, and organize teamwork through GitHub.

2 SYSTEM ARCHITECTURE

The application follows the MVVM pattern, which separates presentation from data handling and preserves state across configuration changes. It has four layers: the Compose interface, a ViewModel, a repository, and the Room database. Events flow down to the database, and state flows back up as Kotlin Flow.

The data layer consists of a Task entity (title, description, priority, due date, completion status), a data access object (DAO) with insert, update, delete, and query operations, and a repository that hides storage details. Queries return a Flow, so database changes reach the interface automatically. The ViewModel exposes the tasks as a StateFlow, combines them with the selected filter (all, active, or completed), and runs actions in coroutines to keep the main thread free.

3 IMPLEMENTATION

Screens are built from small reusable composables that receive state and callbacks as parameters (state hoisting). A Material 3 theme provides light and dark color schemes, and the task list uses an adaptive grid that adjusts to the screen width. Because data is stored on the device, the application works offline.

Navigation Compose connects three screens: the task list (filter chips, completion checkbox, delete, and an add button), the add/edit form (priority selector and title validation), and the task details view. One form serves both adding and editing through a navigation argument, and a Share action sends the task through an implicit ACTION_SEND Intent. The code favors val over var and uses lambdas for callbacks.

4 TEAM CONTRIBUTIONS

Each member owned a complete part of the system (Table 1), developed it on a separate Git branch, and merged it through a reviewed pull request.

Table 1: Division of work among team members

Member	Responsibility
1	Room database: entity, DAO, and database class
2	Repository and ViewModel: state, filtering, and actions
3	Task list screen, filters, and responsive layout
4	Add/edit and details screens, validation, and share Intent
5	Project setup, navigation, theme, integration, review, and documentation

5 CONCLUSION AND FUTURE WORK

The project delivers an offline-capable to-do list application built on Compose, Room, and a clean MVVM structure. Future work includes reminders, categories, search, and cloud synchronization.

REFERENCES

[1] Android Developers. Jetpack Compose. https://developer.android.com/compose

[2] Android Developers. Save data in a local database using Room. https://developer.android.com/training/data-storage/room

[3] Android Developers. Navigation. https://developer.android.com/guide/navigation

[4] Android Developers. ViewModel overview. https://developer.android.com/topic/libraries/architecture/viewmodel

[5] Kotlin Documentation. Asynchronous Flow. https://kotlinlang.org/docs/flow.html